

Proxies & CDNs

Phase 1 · Fundamentals · Estimated time: 1 hour

1. Concept From First Principles

A **proxy** is any server that sits between a client and the ultimate destination, forwarding requests on someone's behalf. Which side it represents determines whether it's called a forward proxy or a reverse proxy — a distinction that trips up a surprising number of experienced engineers in interviews.

Forward proxy vs. reverse proxy

A **forward proxy** sits in front of *clients* and forwards their requests out to the internet on their behalf — the destination server sees the proxy, not the real client. Common uses: corporate networks blocking certain sites, or anonymizing a client's identity.

A **reverse proxy** sits in front of *servers* and forwards incoming client requests to one of several backend servers — the client sees the proxy, not the real backend. This should sound familiar: a load balancer (Chapter 9) is essentially a specialized reverse proxy. Reverse proxies also commonly handle TLS termination (decrypting HTTPS so backend servers don't each need certificates), compression, and caching responses.

Backend engineer analogy

Nginx sitting in front of your PHP-FPM or Node app, terminating SSL and forwarding requests to your app process, is a reverse proxy — you've deployed one for years, possibly without naming it as such. A forward proxy is closer to a VPN or a corporate web filter — it represents the client, not the server.

It's also worth being clear about what reverse proxies and CDNs share versus where they diverge: both cache and both sit between the client and the "real" origin, but a reverse proxy is typically one logical location close to your own servers, while a CDN is deliberately spread across many physical locations close to your *users*, wherever in the world they happen to be. In practice, a production system often layers both — a CDN for global static content, and a reverse proxy at the origin for routing, TLS termination, and any caching a CDN doesn't cover.

CDN — caching content close to the user

A **Content Delivery Network (CDN)** is a globally distributed network of reverse-proxy-like servers (often called edge servers or Points of Presence, PoPs) that cache and serve static content — images, videos, CSS, JS, sometimes even whole cacheable API responses — from a location physically close to the user, instead of every request traveling back to a single origin server. Cloudflare, Akamai, and AWS CloudFront are common providers.

The core win is latency: a user in Riyadh fetching an image from a CDN edge node in the UAE is far faster than that same request traveling all the way to an origin server in, say, Singapore or the US. CDNs also absorb huge amounts of traffic away from your origin servers entirely, which is both a performance and a cost benefit.

One operational detail worth knowing: CDN caching introduces its own invalidation problem, just like Chapter 8's application cache. If a product's image is updated, the CDN's edge copies across dozens of global locations don't know that instantly — teams handle this either by setting short TTLs on highly-changeable assets, or by explicitly "purging" (invalidating) a specific cached path across the CDN's network the moment the source file changes. A common, simpler workaround is versioning the URL itself (e.g., appending a hash: **image.jpg?v=8f2a1**), so a changed file is automatically a cache miss under a brand-new URL, without needing to purge anything.

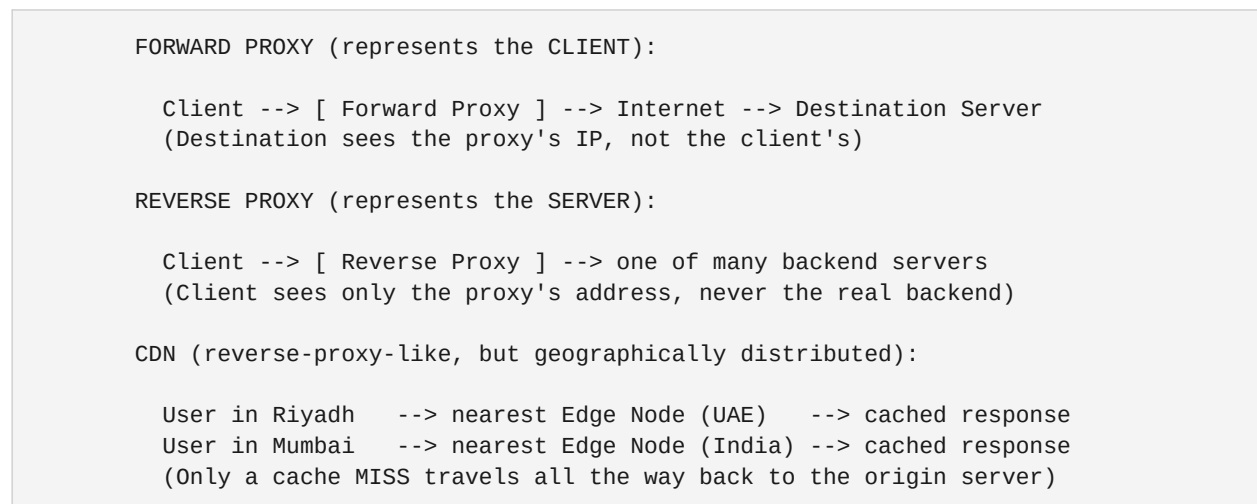
2. Real-World Example / Case Study

Noon and Talabat serve product images and app assets through a CDN, so a shopper in Jeddah loads images from a nearby edge node instead of a single origin data center — shaving hundreds of milliseconds off page load, which directly affects conversion rates in e-commerce. Streaming platforms and food-delivery apps with promotional videos rely on CDNs even more heavily, since video is the most bandwidth-hungry content type.

Razorpay's public-facing checkout widget is typically served through a reverse proxy/CDN layer that also provides a first line of defense against DDoS attacks — absorbing and filtering malicious traffic before it ever reaches the actual payment-processing servers.

A third example: many Indian enterprises use forward proxies internally to filter and log outbound employee internet traffic for compliance reasons — a completely different use case from Noon's CDN, but built on the exact same "proxy sits between a client and the destination" primitive. Recognizing that forward and reverse proxies share a mechanism but serve opposite purposes is precisely the kind of distinction interviewers probe to check real understanding versus memorized definitions.

3. Diagram



4. Trade-offs Table

Component	Solves	Doesn't solve
Forward proxy	Client anonymity, access control, outbound filtering	Doesn't help scale or protect your own servers

Component	Solves	Doesn't solve
Reverse proxy	TLS termination, routing, hides backend topology	Doesn't reduce geographic latency by itself
CDN	Geographic latency, origin traffic offload, DDoS absorption	Not useful for highly dynamic, per-user, uncacheable data

5. Common Interview Questions

Q1: "What's the difference between a forward proxy and a reverse proxy?"

Model approach: State plainly which side each one represents ("forward proxy represents the client, reverse proxy represents the server"), then give one example use case of each so the answer isn't purely definitional.

Q2: "Would a CDN help speed up our /checkout API? Why or why not?"

Model approach: Explain that CDNs are built for cacheable, mostly-static content shared across many users; a checkout API response is dynamic and unique per user/session, so caching it at the edge is inappropriate (or requires very careful, narrow caching rules). Contrast with product images or a public product listing page, which are excellent CDN candidates.

Q3: "How would you handle updating an image that's already cached across a global CDN?"

Model approach: Offer two options and their trade-off — actively purge/invalidate the specific path across all edge nodes (immediate but an extra operational step every time), or version the asset's URL so an update is automatically a new, uncached URL (simpler, no purge step needed, but requires updating every reference to that asset's URL).

6. Practice Exercises

- 1 List 3 pieces of content in a food delivery app that belong on a CDN, and 3 that should never be cached at the edge.
- 2 Explain, without notes, why a reverse proxy handling TLS termination simplifies certificate management for a fleet of 50 backend servers.
- 3 Describe what happens on a CDN cache MISS versus a cache HIT, end to end.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"What Is A CDN? How Does It Work?"	Clear, focused explanation of CDN mechanics.
English	"Caching Levels Explained: Browser, CDN, Reverse Proxy & Application Caching"	Ties CDN caching into the broader caching picture from Chapter 8.
Hindi	"Proxy Server [Hindi]" playlist	Hindi-language coverage of proxy server concepts and configuration.

Language	Video & Channel	Why it helps
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Covers CDN/proxy fundamentals within the broader Hindi roadmap.

8. Key Takeaways

- A forward proxy represents the client; a reverse proxy represents the server.
- Load balancers are a specialized form of reverse proxy.
- Reverse proxies commonly handle TLS termination, routing, and hiding backend topology from clients.
- A CDN is a geographically distributed set of edge caches that serve static/cacheable content close to the user.
- CDNs reduce latency and offload traffic from origin servers, but aren't suited for highly dynamic, per-user data.
- Deciding what's cacheable (CDN-friendly) vs. dynamic (not) is a recurring, practical interview judgment call.